

Model Context Protocol

Open Protocol for LLM & Tools Interaction

Problem Domain

- So far, we have seen how to code with LLMs
- How Langchain facilitates building Gen AI Application
- Custom tools and applications integrated with LLMs to build multi-layered LLM Based Solutions

BUT ...

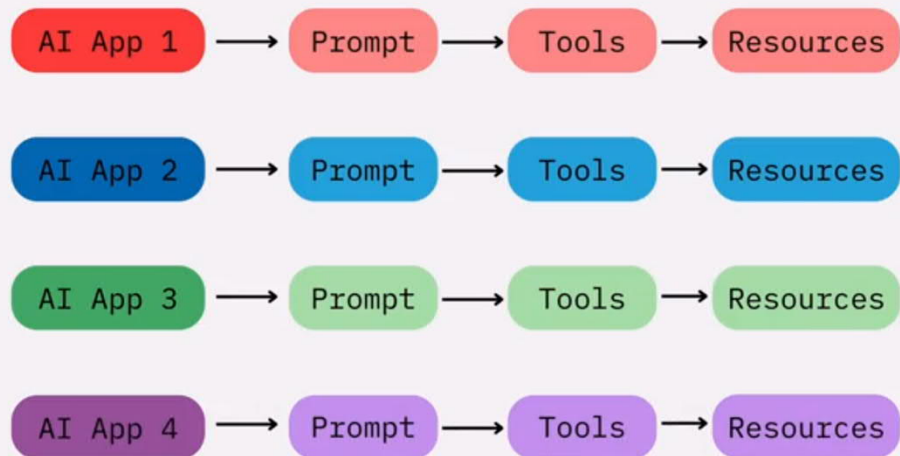
- Tokens are the currency that we burn in building such applications
- Tool definitions alone can consume huge number of tokens before the agent even reads the user's request.
- Tight Coupling. Everyone has to define tools and functionality everytime!
- Poor interoperability.

<https://www.anthropic.com/engineering/code-execution-with-mcp>

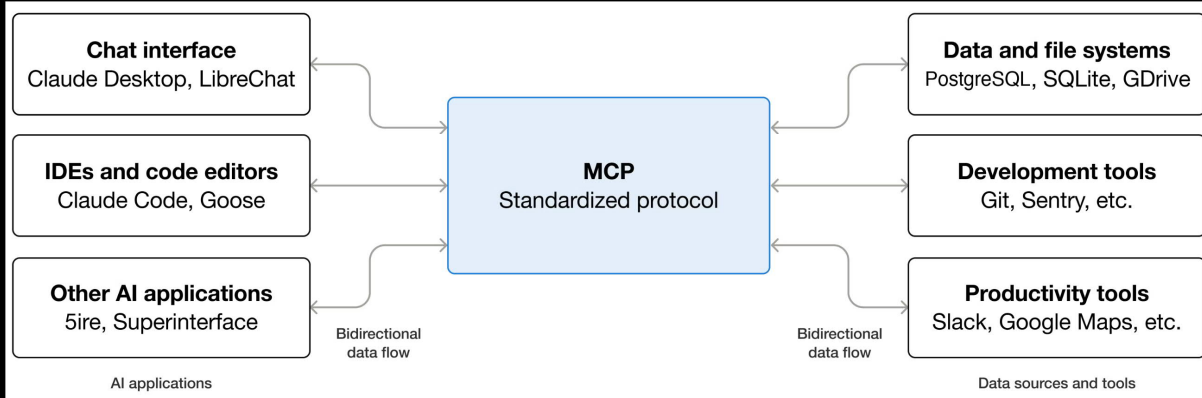
The Model Context Protocol (MCP) is an open standard for connecting AI agents to external systems. Connecting agents to tools and data traditionally requires a custom integration for each pairing, creating fragmentation and duplicated effort that makes it difficult to scale truly connected systems. MCP provides a universal protocol—developers implement MCP once in their agent and it unlocks an entire ecosystem of integrations.

Since launching MCP in November 2024, adoption has been rapid: the community has built thousands of MCP servers, SDKs are available for all major programming languages, and the industry has adopted MCP as the de-facto standard for connecting agents to tools and data.

Problem Domain (cont...)



What is the Model Context Protocol



- MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.
- AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.
- Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.

<https://modelcontextprotocol.io/docs/getting-started/intro>

What is the Model Context Protocol (MCP)?

MCP is an open protocol that enables seamless integration between **LLM applications** and your **tools & data sources**

USB C for AI Applications

APIs

Standardize how **web applications** interact with the **backend**:

- Servers
- Databases
- Services

LSP (Language Server Protocol)

Standardizes how **IDEs** interact with **language-specific tools**:

- Code navigation
- Code analysis
- Code intelligence

MCP

Standardizes how **AI applications** interact with **external systems**:

- Prompts
- Tools
- Data & resources
- Sampling

We know we'll never be able to fully customize the product to the precise requirement of your workflows. That's why we want to give you the ability to customize Claude to your needs. MCP is a new protocol we are pioneering to allow Claude to easily interact with external systems

To help understand MCP's role, let's look at two similar protocols that transformed software development:

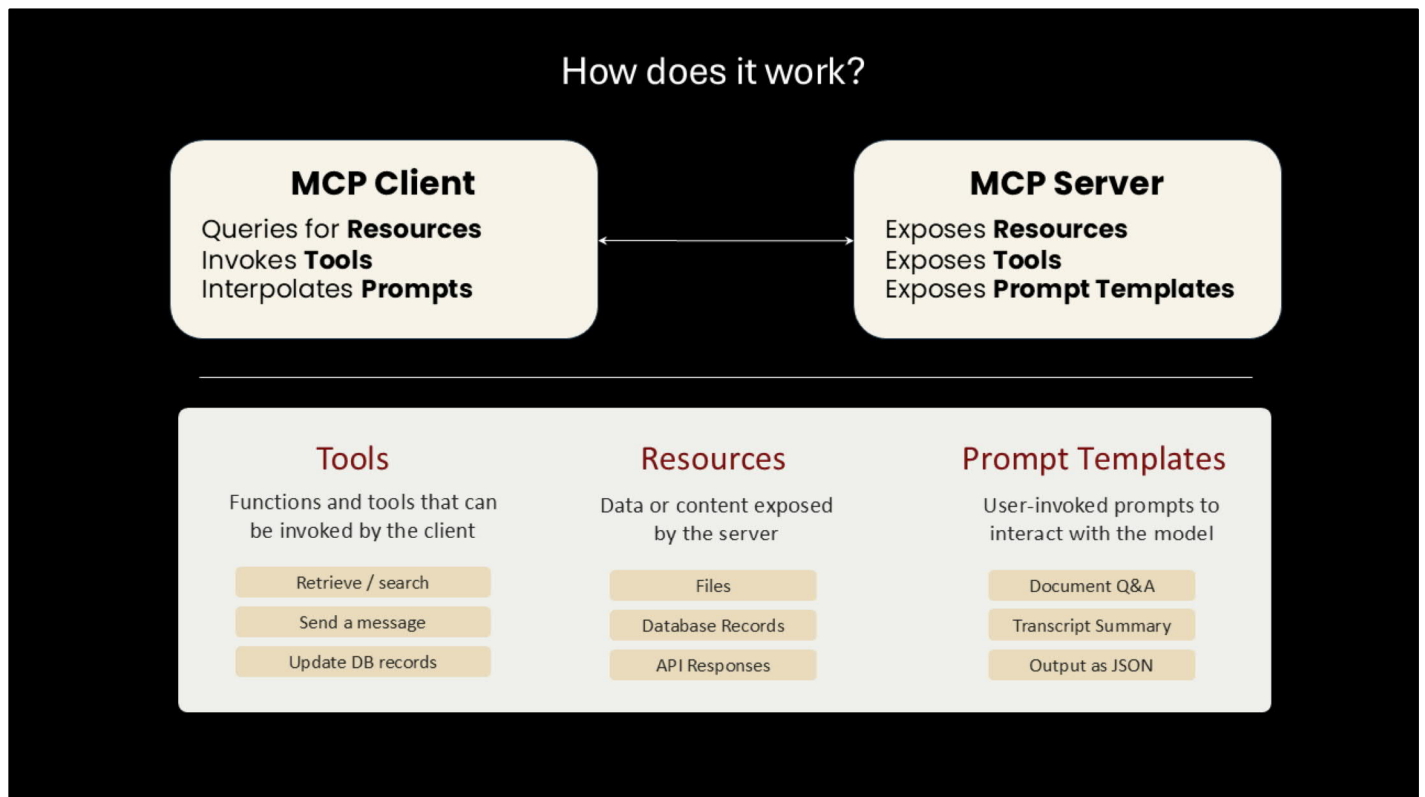
- APIs standardized how web applications talk to backends and databases. They created a common language for data exchange.
- LSP (Language Server Protocol) standardized how IDEs interact with programming language tools. Write one language server, it works in VS Code, Vim, or any LSP-compatible editor.

MCP does the same thing for AI applications by standardizing three critical elements:

- Prompts: Reusable templates for common tasks
- Tools: Functions that let the AI take actions
- Data & Resources: Ways to access and understand information

The key benefits of MCP:

- **Customize any AI app** (Claude or anything you're building yourself) with everything from local files on your computer to cloud applications and internal knowledge bases.
- **Dramatically reduces development effort** through a 'write once, work everywhere'. Just like you write one REST API and any web client can use it, you write one MCP server and any AI application can use it.
- **Clearly separate development**, similar to how we separate front-end from back-end work. Teams can specialize in either building MCP clients or servers.



- **Client app** (typically local today; cloud-hosted coming) → runs the LLM, decides when to call a tool.
- **MCP servers** → surface capabilities the model can invoke.

What a server can expose

Tools – most common (CRUD on data, Git ops, CRM actions...).

Resources – raw data endpoints / RAG queries the model can pull.

Prompt templates – reusable system or task prompts the client can insert.

Sampling calls – server can ask the client/LLM to do work on its behalf (rare but powerful).

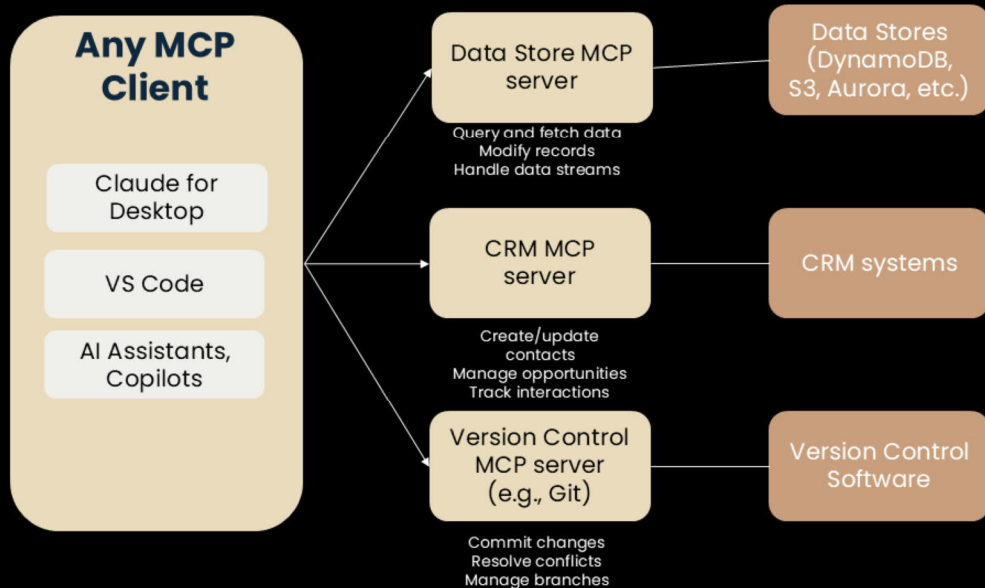
Current State of the ecosystem

- Lots of tool-only servers already
- Very **few use Resources & Prompt Templates**—ripe area to explore / contribute.

Key Takeaway

Don't just build another tool server; try exposing curated RAG queries or prompt packs—low competition, high impact.

MCP: Standardized AI Development



Client column (LLM lives here)

→ any of these can host the model & decide when to fire a tool call.

Server column (tools exposed)

Back-end targets (far right)

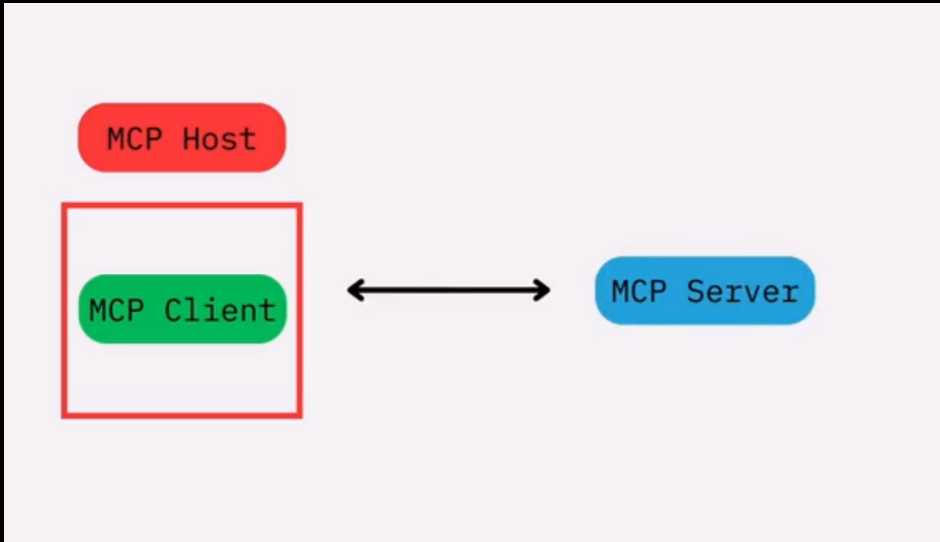
Flow to emphasize

Client ⇒ picks tool ⇒ sends request to server ⇒ server bridges to real system ⇒ result back to model.

Key takeaway

Swap clients *or* servers independently—MCP is the USB-C that keeps them interoperable.

MCP Architecture



The key participants in the MCP architecture are:

MCP Host: The AI application that coordinates and manages one or multiple MCP clients

MCP Client: A component that maintains a connection to an MCP server and obtains context from an MCP server for the MCP host to use

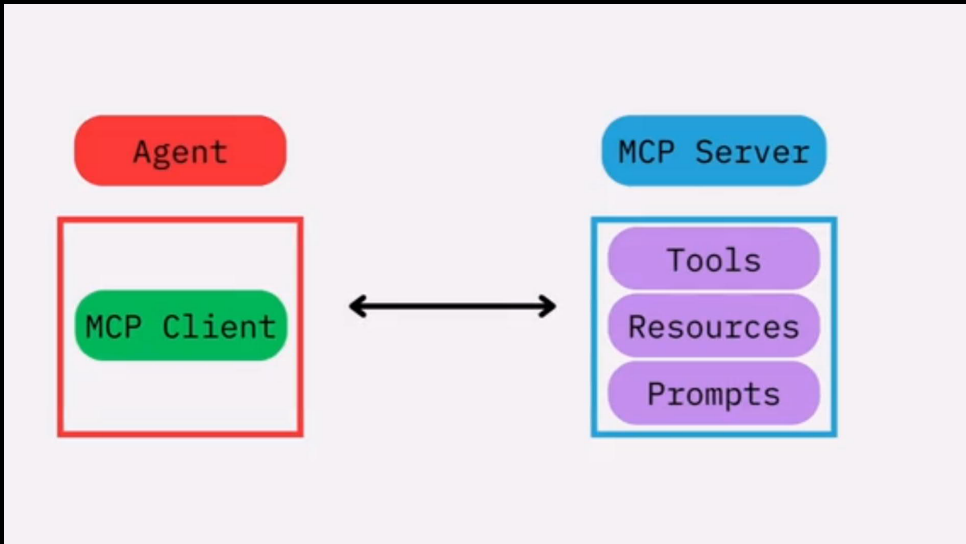
MCP Server: A program that provides context to MCP clients

For example: Visual Studio Code acts as an MCP host. When Visual Studio Code establishes a connection to an MCP server, such as the [Sentry MCP server](#), the Visual Studio Code runtime instantiates an MCP client object that maintains the connection to the Sentry MCP server. When Visual Studio Code subsequently connects to another MCP server, such as the [local filesystem server](#), the Visual Studio Code runtime instantiates an additional MCP client object to maintain this connection.

Note that **MCP server** is a reference to the application/program that serves context data, regardless of where it runs. MCP servers can execute locally or

remotely. For example, when Claude Desktop launches the [filesystem server](#), the server runs locally on the same machine because it uses the STDIO transport. This is commonly referred to as a “local” MCP server. The official [Sentry MCP server](#) runs on the Sentry platform, and uses the Streamable HTTP transport. This is commonly referred to as a “remote” MCP server.

MCP Architecture



MCP Architecture: Transport Layer

- In MCP, a "transport" is simply the pipe that carries messages back and forth. Since MCP is based on JSON-RPC 2.0, the messages themselves are just structured text, but the way that text travels depends on the transport method used.
- **stdio** and **SSE** (Server-Sent Events) are the two primary ways this happens.

<https://modelcontextprotocol.io/docs/learn/architecture>

Participants

MCP follows a client-server architecture where an MCP host — an AI application like [Claude Code](#) or [Claude Desktop](#) — establishes connections to one or more MCP servers.

The MCP host accomplishes this by creating one MCP client for each MCP server. Each MCP client maintains a dedicated connection with its corresponding MCP server.

The transport layer manages communication channels and authentication between clients and servers. It handles connection establishment, message framing, and secure communication between MCP participants.

Local MCP servers that use the STDIO transport typically serve a single MCP client, whereas remote MCP servers that use the Streamable HTTP transport

will typically serve many MCP clients.

MCP Client/Server Communication: stdio

- **stdio → Standard Input/Output**
- This is the "local" transport. When an MCP client (like a desktop AI app) wants to use a tool, it starts the MCP server as a **child process**.
- **The Pipe:** It sends JSON data to the server's stdin and reads responses from the server's stdout.
- **Why use it?** It's incredibly fast because there is no network overhead. It's also very secure because the server only exists as long as the client is running it.

Stdio transport: Uses standard input/output streams for direct process communication between local processes on the same machine, providing optimal performance with no network overhead.

Langchain MCP Library

```
from langchain_mcp_adapters.client
    import MultiServerMCPClient

client = MultiServerMCPClient(
    {
        "local_server": {
            "transport": "stdio",
            "command": "python",
            "args": ["<mcp_server_file_path>.py"],
        }
    })
```

MCP Client/Server Communication: SSE

- This is the "remote" transport, built on top of **HTTP**.
- **The Pipe:** The client opens a long-running connection to an HTTP endpoint to receive a stream of messages from the server. To send messages to the server, the client makes standard HTTP POST requests.
- **Why use it?** This allows your AI to talk to tools running in the cloud or on a different server entirely.

Uses HTTP POST for client-to-server messages with optional Server-Sent Events for streaming capabilities.

This transport enables remote server communication and supports standard HTTP authentication methods including bearer tokens, API keys, and custom headers.

MCP recommends using OAuth to obtain authentication tokens.

MCP Architecture: Data Layer Protocol

- MCP uses **JSON-RPC 2.0** as its foundational language. This is a lightweight protocol that allows one system to "call" a function on another system and get a structured result back.
- Every action (like "list tools" or "read resource") follows a strict structure:
 - **ID:** Matches a specific response to its original request.
 - **Method:** The specific action being requested (e.g., tools/call).
 - **Params:** The input data or arguments for that action.
- Allows the server or client to send "one-way" updates (like a progress report) without requiring a formal response.
- Provides standardized error codes so the AI knows if a failure was due to a connection issue, a missing tool, or invalid inputs.

A core part of MCP is defining the schema and semantics between MCP clients and MCP servers. Developers will likely find the data layer — in particular, the set of **primitives** — to be the most interesting part of MCP. It is the part of MCP that defines the ways developers can share context from MCP servers to MCP clients.

MCP uses **JSON-RPC 2.0** as its underlying RPC protocol. Client and servers send requests to each other and respond accordingly. Notifications can be used when no response is required.

<https://www.jsonrpc.org/>

JSON-RPC is a stateless, light-weight remote procedure call (RPC) protocol. Primarily this specification defines several data structures and the rules around their processing. It is transport agnostic in that the concepts can be used within the same process, over sockets, over http, or in many various message passing environments. An rpc call is represented by sending a Request object to a Server.

The **Request** object has the following members:

jsonrpc: A String specifying the version of the JSON-RPC protocol. MUST be exactly "2.0".

method: A String containing the name of the method to be invoked. Method names that begin with the word rpc followed by a period character (U+002E or ASCII 46) are reserved for rpc-internal methods and extensions and MUST NOT be used for anything else.

params: A Structured value that holds the parameter values to be used during the invocation of the method.

id: An identifier established by the Client that MUST contain a String, Number, or NULL value if included. If it is not included it is assumed to be a notification.

Notifications: The protocol supports real-time notifications to enable dynamic updates between servers and clients. For example, when a server's available tools change—such as when new functionality becomes available or existing tools are modified—the server can send tool update notifications to inform connected clients about these changes. Notifications are sent as JSON-RPC 2.0 notification messages (without expecting a response) and enable MCP servers to provide real-time updates to connected clients.

MCP Server Primitives

- In MCP, the data layer organizes information into three main "Primitives."
 - Think of these as the three types of things an AI can "see" or "do" through the protocol
- **Prompts** 💡 : Pre-defined templates that help the LLM understand how to interact with a specific set of data.
- **Resources** 📄 : Read-only data (like a file's contents or a database row).
 - Structured data or content that provides additional context to the model
- **Tools** 🛠️ : Executable functions that allow models to perform actions or retrieve information

MCP Server Primitives

- Each primitive type has associated methods for discovery (`*/list`), retrieval (`*/get`), and in some cases, execution (`tools/call`).
- Clients will use the `*/list` methods to discover available primitives.
 - For example, a client can first list all available tools (`tools/list`) and then execute them.
 - This design allows listings to be dynamic.
- Example: Consider an MCP server that provides context about a database.
 - It can expose tools for querying the database,
 - a resource that contains the schema of the database, and
 - a prompt that includes few-shot examples for interacting with the tools.

MCP Server Primitives

Primitive	Control	Description	Example
Prompts	User-controlled	Interactive templates invoked by user choice	Slash commands, menu options
Resources	Application-controlled	Contextual data attached and managed by the client	File contents, git history
Tools	Model-controlled	Functions exposed to the LLM to take actions	API POST requests, file writing

In **MCP (Model Context Protocol)**, slash commands are a **UI-level, structured way for a client** (like a chat app, IDE, or Copilot UI) to expose predefined actions that are backed by MCP server capabilities.

They are **not part of the MCP wire protocol itself** — they are a **convention implemented by MCP clients** on top of MCP tools, prompts, or resources.

A **shortcut command (starting with /)** that the user types in a chat or input box, which the client translates into a **specific MCP action**, such as invoking a tool, loading a prompt, or fetching a resource.

Example from a user's point of view:

/summarize

/debug

/explain

From MCP's point of view:

→ invoke a tool

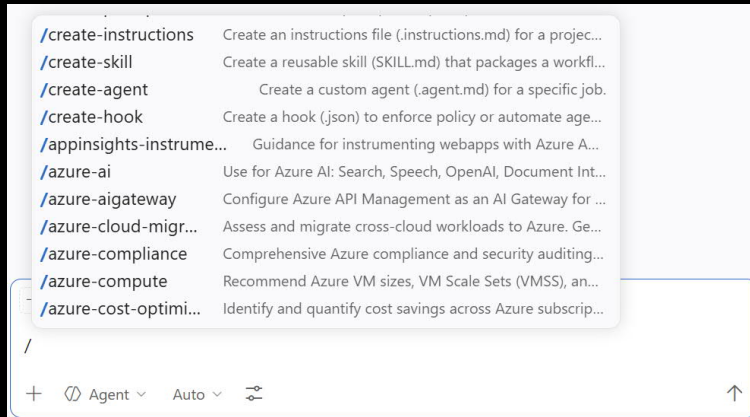
→ load a predefined prompt

→ call a server capability

Server Primitives: Prompts

Prompts (slash commands)

- Prompts are designed to be **user-controlled**; they are exposed from servers to clients with the intention of the user being able to explicitly select them for use.



Prompts available in VSCode based on the available MCP Servers (Copilot, mslearn, Git, Azure, etc.)

<https://modelcontextprotocol.io/specification/2025-11-25/server/prompts>

Server Primitives: Resources

Resources

- **MCP Resources** are **read-only, URI-addressable context** (files, schema, configs, logs, snapshots) that clients can fetch and attach to model context.
- The MCP spec defines `resources/list` and `resources/read` for discovery and retrieval.

Fast MCP: <https://gofastmcp.com/servers/resources>

Example: “Data Platform Helper” MCP server

We’ll expose 3 resources:

`data://config` → returns JSON config (read-only, safe subset)

`schema://{table}` → returns a dynamic schema string for a table

`runlog://latest` → returns latest pipeline run log excerpt

These are classic “context” artifacts that an LLM uses to answer questions like:

“What columns does `silver_orders` have?”

“What is the current ingestion mode?”

“Why did the last run fail?”

MCP SDKs (As of March 2026)

SDK	Repository	Tier
TypeScript	modelcontextprotocol/typescript-sdk	Tier 1
Python	modelcontextprotocol/python-sdk	Tier 1
C#	modelcontextprotocol/csharp-sdk	Tier 1
Go	modelcontextprotocol/go-sdk	Tier 1
Java	modelcontextprotocol/java-sdk	Tier 2
Rust	modelcontextprotocol/rust-sdk	Tier 2
Swift	modelcontextprotocol/swift-sdk	Tier 3
Ruby	modelcontextprotocol/ruby-sdk	Tier 3
PHP	modelcontextprotocol/php-sdk	Tier 3
Kotlin	modelcontextprotocol/kotlin-sdk	TBD